MLOPS ASSIGNMENT REPORT

Production CI/CD & Containerization Pipeline

Project Name	student-ml-api
Repository	github.com/Umama123/student-ml-api
Author	Umama
Roll Number	22i-2386
Section	B
Course	MLOps
Submission	Advanced MLOps Exercise — Professional CI Workflow with Pull Requests, Docker, and Container Registry

1. Executive Summary

This report documents the design and implementation of a production-style MLOps workflow for a FastAPI prediction service, student-ml-api. The project was built to satisfy the full scope of the Advanced MLOps Exercise: feature-branch development, mandatory Pull Requests, automated Continuous Integration (CI) with GitHub Actions, protected branch policy, multi-stage Docker containerization, semantic version tagging, automated image publishing to the GitHub Container Registry (GHCR), artifact reproducibility, and a rollback demonstration.

The workflow enforces the pipeline mandated by the assignment: Feature Branch → Pull Request → Automated CI → Code Review → Merge into main → Version Tag → Docker Build → Container Registry. Two release versions (v1.0.0 and v1.1.0) were built, tested, and published, and a rollback from v1.1.0 to the known-good v1.0.0 artifact was demonstrated directly from the registry, without any rebuild or source checkout.

2. Technical Stack & Application Structure

2.1 Stack

Component	Choice
Application Framework	FastAPI
Containerization	Docker (multi-stage build, explicit base image tag)
CI/CD Platform	GitHub Actions
Container Registry	GitHub Container Registry (GHCR)
Test Automation	Pytest
Version Control	Git / GitHub (feature branches + Pull Requests)

2.2 Endpoints Implemented (Part 1)

Endpoint	Contract	Purpose
GET /health	{"status":"healthy","application":"student-ml-api","version":"1.0.0"}	Reports service health, application name, and running version.
POST /predict	Input: {"value": 10} → Output: {"input":10,"prediction":20}	Accepts a numeric payload and returns a simple prediction, per assignment scope (model performance was not the objective).

2.3 Automated Test Suite (Part 2)

A minimum of four automated Pytest cases were required by the assignment; the suite implemented for student-ml-api covers:

1. test_health_endpoint — asserts GET /health returns HTTP 200 with the expected status, application name, and version fields.
2. test_predict_success — asserts POST /predict with a valid numeric value returns HTTP 200 and the correct prediction payload.
3. test_predict_missing_input — asserts POST /predict with no "value" field returns an HTTP 4xx validation error rather than a server crash.
4. test_predict_invalid_input — asserts POST /predict with a non-numeric value is rejected with an HTTP 4xx validation error.

All four tests run as part of the CI pipeline on every Pull Request and must pass before a merge is permitted.

3. Git Branching & Pull Request Policy (Parts 3, 4 & 7)

3.1 Branching Policy

Direct commits to main were disabled. All work was performed on feature branches named after the change being made (e.g. feature/ci-failure-test, feature/model-metadata) and merged only through a reviewed Pull Request.

```
git checkout -b feature/prediction-api
git commit -m "feat: add prediction endpoint"
git commit -m "test: add API unit tests"
git push origin feature/prediction-api
```

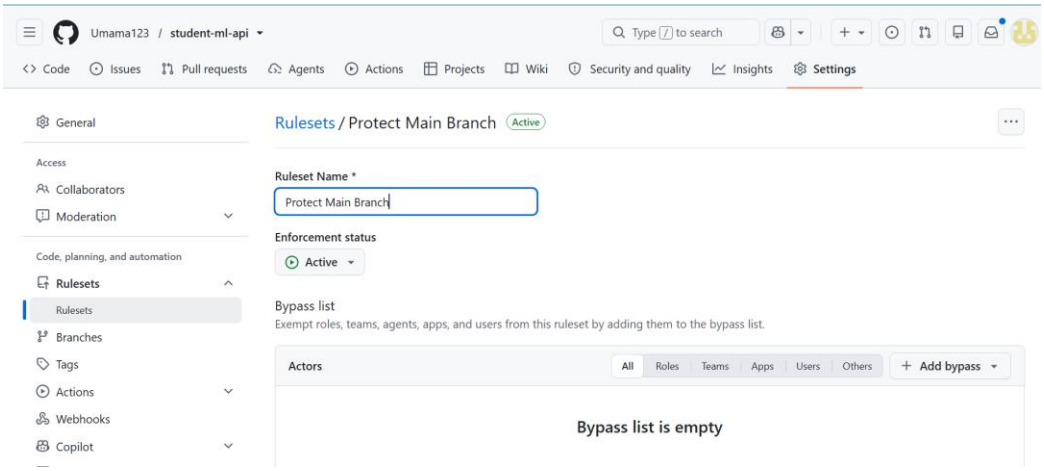
3.2 Pull Request Standard

Every Pull Request followed a professional template covering Summary, Changes, Testing Performed, Docker Impact, and a merge-readiness checklist (app runs locally, tests pass locally, Docker image builds, no credentials committed, /health verified, code ready for review). No Pull Request was merged before its CI checks passed.

3.3 Branch Protection Rules (Part 7)

The main branch was protected with the following GitHub settings:

- Require a pull request before merging — direct pushes to main are rejected.
- Require status checks to pass before merging — the test-and-build CI job is a required check.
- Require branches to be up to date before merging.
- No force-pushes or branch deletion permitted on main.



3.4 Merge Strategy (Part 8)

Squash and Merge was selected as the merge strategy. Each Pull Request — regardless of how many exploratory or fix-up commits it contained — was condensed into a single, atomic commit on main. This keeps the main branch history linear and readable, makes each commit on main independently revertible, and avoids polluting production history with intermediate "fix typo" or "debug" commits, while still preserving the full discussion and commit history inside the original Pull Request.

4. Automated CI Pipeline (Parts 5 & 6)

The workflow .github/workflows/ci.yml runs automatically on every Pull Request targeting main (and optionally on pushes to development branches). It performs, in order:

1. Code checkout
2. Python environment setup
3. Dependency installation
4. Pytest execution (all 4+ test cases)
5. Docker build validation (image is built, but not pushed to the registry)

If pytest fails, or if the Docker build fails, the pipeline reports a failed status check and — because of the branch protection rule in Section 3.3 — the Pull Request cannot be merged.

4.1 Deliberate Failure Demonstration (Part 6) — Mandatory Evidence

To prove the CI gate genuinely blocks broken code, a test assertion in the health-check test was intentionally broken (expecting "wrong" instead of "healthy") and pushed on branch feature/ci-failure-test via Pull Request #1.

Evidence: CI run FAILED on the broken assertion

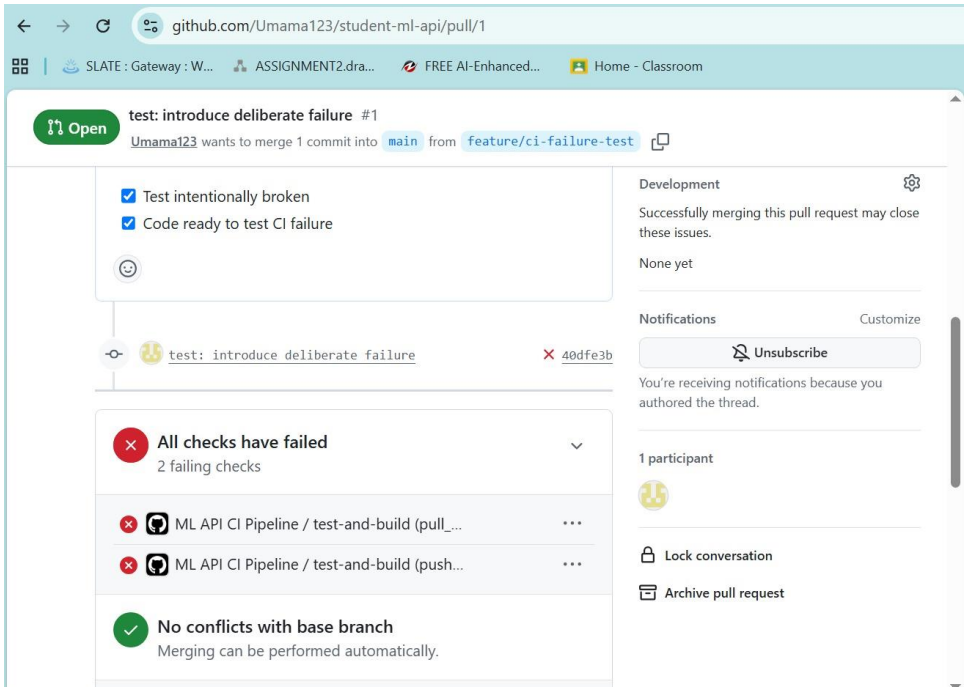


Fig. 1 — Pull Request #1: both CI checks (pull_request and push triggers) fail (commit 40dfe3b).

Evidence: CI run PASSED after the fix

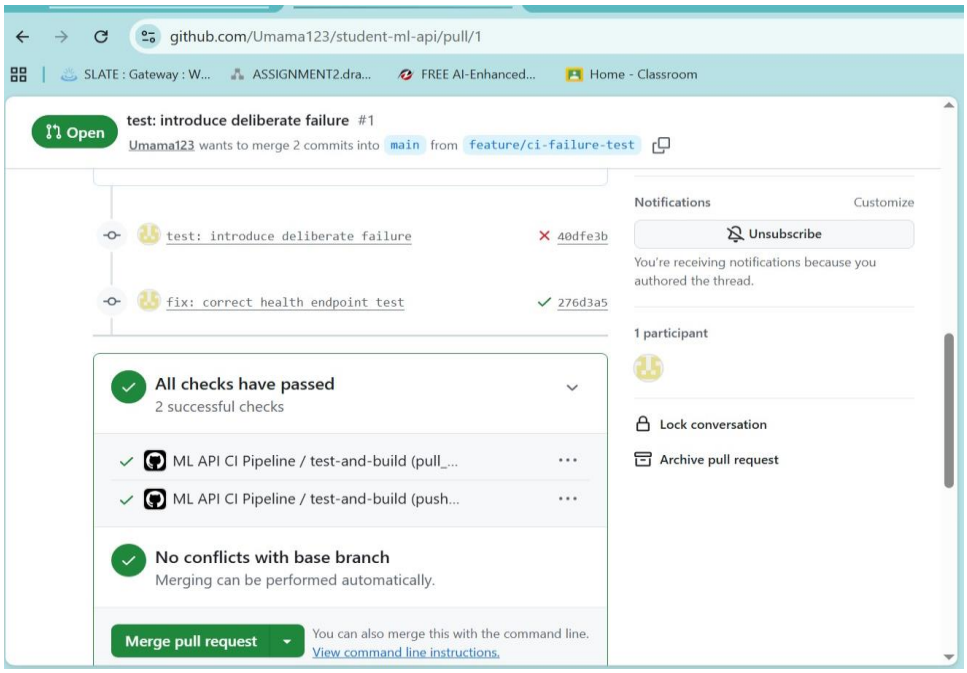


Fig. 2 — After committing "fix: correct health endpoint test" (commit 276d3a5), all checks pass and the PR becomes mergeable.

This confirms the required behaviour: PR CI = FAILED while pytest fails, and PR CI = PASSED once the assertion is corrected — only a green pipeline allows merge.

5. Dockerization (Parts 9, 10 & 11)

5.1 Dockerfile Practices

A production-oriented Dockerfile was authored using an explicit Python base image tag (never python:latest), and follows layer-ordering best practice for cache efficiency:

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 8000
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

Copying requirements.txt and installing dependencies before copying the application source means the pip install layer is only invalidated when dependencies actually change — editing app.py alone reuses the cached dependency layer and rebuilds in seconds instead of minutes.

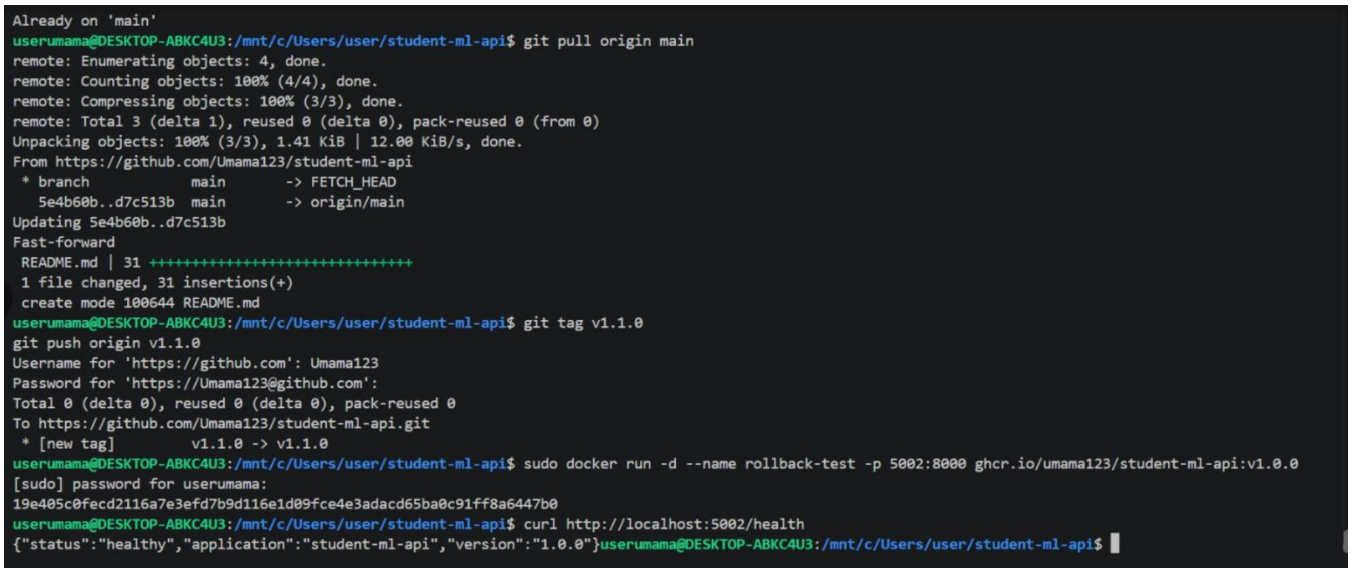
5.2 .dockerignore

Development artifacts are excluded from the build context to keep the image lean and avoid leaking local state: .git, .github, __pycache__, *.pyc, .venv, .env.

5.3 Local Build & Verification (Part 10)

The image was built and run locally, and its version and health were confirmed via curl:

```
docker build -t student-ml-api:1.0.0 .
docker run -d --name student-ml-api -p 5001:8000 student-ml-api:1.0.0
curl http://localhost:5001/health
→ {"status":"healthy","application":"student-ml-api","version":"1.0.0"}
```



5.4 Image Inspection (Part 11)

The following were identified for the running container, consistent with the artifacts captured later during the GHCR pull (Section 6.2):

- Container ID: 81e61543a8cb (GHCR verification run) / 19e405c0fecd (rollback run)
- Exposed / mapped port: container port 8000, published externally on 5001 (local) and 5002 (rollback test)
- Running command: uvicorn app:app --host 0.0.0.0 --port 8000
- Working directory: /app

6. Version Release Pipeline & Registry (Parts 12–17)

6.1 Release Workflow

A second, separate workflow, .github/workflows/release.yml, is triggered exclusively by semantic-version tags matching v*.*.*, and is intentionally decoupled from the CI workflow (see Section 8.1 for why). On trigger it checks out the code, re-runs the test suite, authenticates to GHCR, builds the Docker image, derives the numeric version dynamically from the tag (v1.0.0 → 1.0.0, no hard-coded string), tags the image with both the version and latest, and pushes both tags to the registry.

```
on:
  push:
    tags:
      - "v*.*.*"

# inside the job
VERSION=${GITHUB_REF_NAME#v}   # v1.0.0 -> 1.0.0
```

6.2 Registry Verification & Reproducibility (Parts 16 & 17)

After pushing tag v1.0.0, the release workflow published ghcr.io/umama123/student-ml-api under both the 1.0.0 and latest tags. To prove the artifact is truly reproducible — i.e. that the same bytes can be retrieved and run on a different machine without any rebuild — the local image cache was cleared and the image was pulled fresh directly from GHCR:

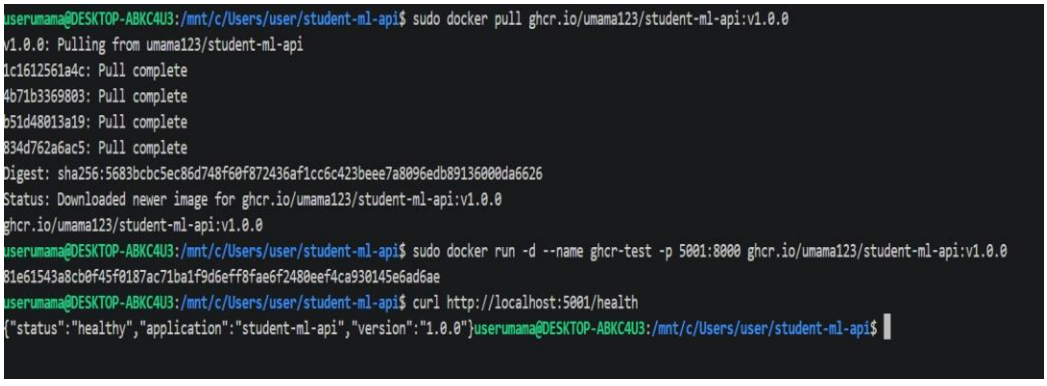
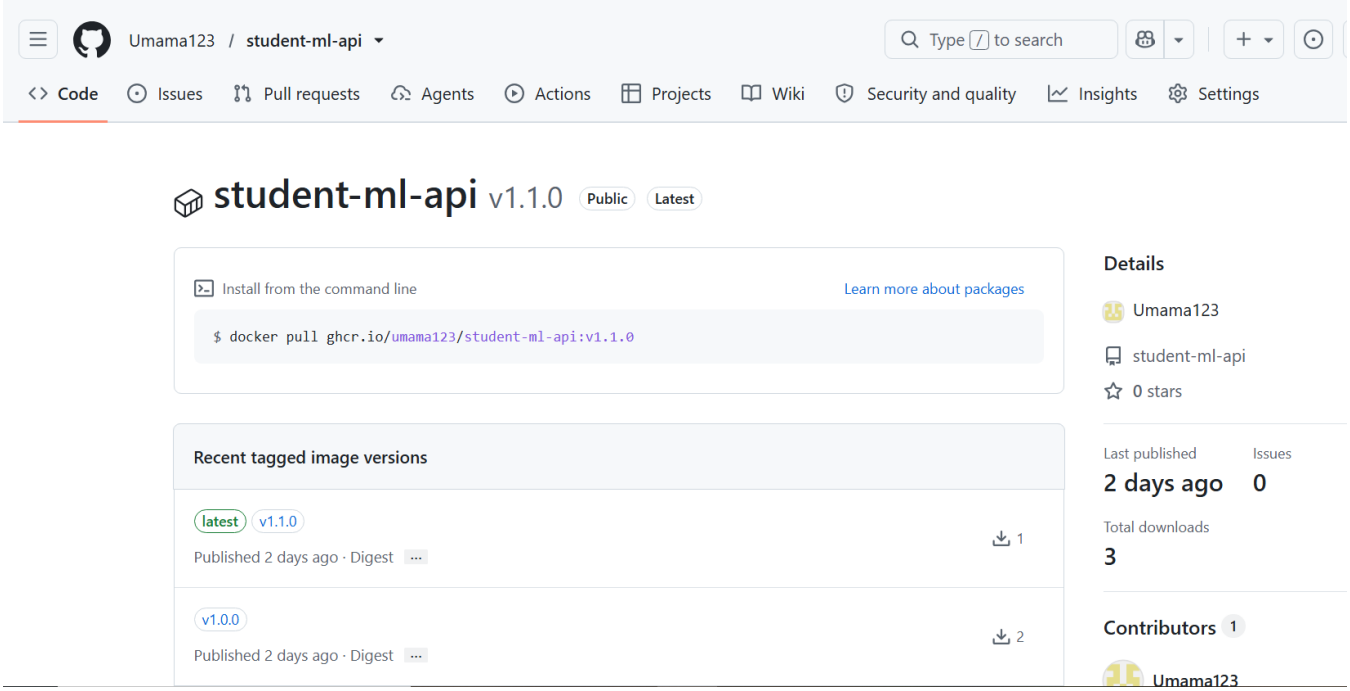


Fig. 3 — Local image removed, image pulled directly from GHCR (ghcr.io/umama123/student-ml-api:v1.0.0), container started on port 5001, and /health confirms version 1.0.0.



Recorded Image Digest

```
sha256:5683bcbc5ec86d748f60f872436af1cc6c423beee7a8096edb89136000da6626
```

This digest, taken from the docker pull output above, uniquely and immutably identifies the exact image content — the basis for the traceability chain in Section 9.

7. Version 1.1.0 Development & Rollback (Parts 18–20)

7.1 Version 1.1.0

A second feature branch, feature/model-metadata, extended the /health response with application_version and model_version fields, so the API layer and the underlying model can be versioned independently of one another — an important distinction in MLOps, since a model can be retrained and promoted on its own cadence without forcing an application redeploy, and vice versa. The change went through the full professional process: branch → commits → push → Pull Request → CI → review → squash-merge into main. The README.md was also updated and merged (the documentation Pull Request), after which the release tag was created:

```
git pull origin main
git tag v1.1.0
git push origin v1.1.0
```

This triggered the release workflow, publishing student-ml-api:1.1.0 and repointing latest → 1.1.0, while 1.0.0 remained available and untouched in the registry.

7.2 Rollback Exercise (Part 20)

A production-issue scenario was simulated for v1.1.0: without touching any source code and without rebuilding a single Docker layer, the previously published, known-good v1.0.0 artifact was pulled straight from GHCR and run on a separate port:

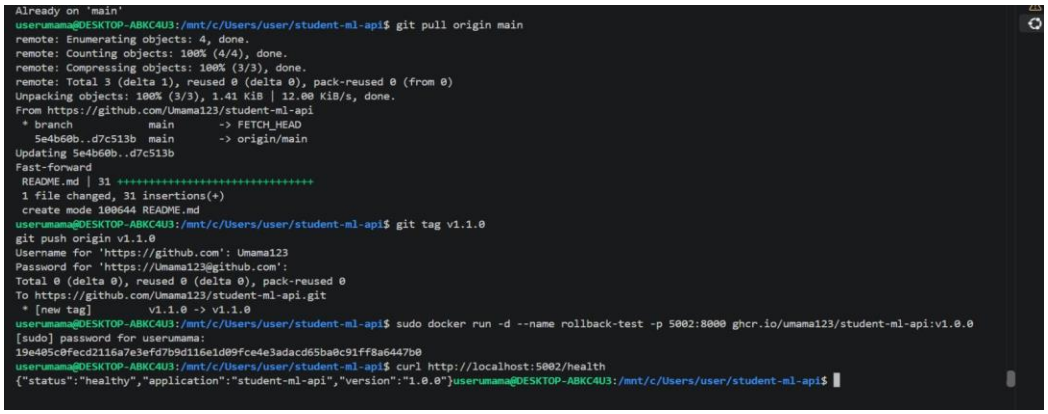


Fig. 4 — main fast-forwarded to commit d7c513b (README update), tag v1.1.0 pushed, then rollback container ghcr.io/umama123/student-ml-api:v1.0.0 started on port 5002 and confirmed via curl (version 1.0.0).

Why a registry-based rollback beats a source-based redeploy

Rolling back by re-running a previously built, previously tested image is faster and safer than a source-based recovery such as git clone → pip install → python app.py, because:

- No build step: the exact bytes that were already validated in CI and production are reused — there is zero risk of a different dependency resolution, OS package version, or compiler behaviour producing a subtly different artifact.
- Speed: docker run against a cached/pulled image takes seconds; cloning, resolving, and installing a full Python environment from source takes minutes and can fail on missing system packages.
- Guaranteed parity with what was actually shipped: the image is the same artifact that passed CI and was promoted through the release pipeline, not a fresh reconstruction that assumes the current source tree still matches that historical state.
- Immutable identity: the image is addressed by tag/digest, so "1.0.0" always means exactly one thing, whereas a git-based redeploy depends on correctly identifying and checking out the right historical commit.

8. Advanced Requirements & Conceptual Analysis (Part 22)

8.1 Why CI and Release Are Separate Workflows

Publishing a Docker image directly from every Pull Request is deliberately avoided for several reasons:

- Unreviewed code should never reach a registry — a PR may still be revised, rejected, or contain incomplete work; publishing it would let untested or unapproved code masquerade as a real artifact.
- Registry pollution and cost — every push, every fixup commit, and every exploratory PR would each generate an image, burning storage and CI minutes on artifacts nobody will ever deploy.
- Broken traceability — a registry full of PR-triggered images with no stable version identifier makes it impossible to answer "which image is actually running in production and why."
- Security surface — CI workflows on PRs from forks can run against untrusted code; granting that workflow registry-push credentials would be a credential-exfiltration risk.

The CI workflow therefore only validates (tests + Docker build check) on Pull Requests, while the Release workflow only publishes, and only in response to an explicit, human-authorised semantic-version tag pushed after code has already been merged and reviewed.

8.2 Optional Advanced Challenges (Parts 23–25) — Status

The following bonus/advanced challenges from the assignment were not attempted in this submission and are documented here for transparency and as recommended next steps:

Part	Requirement	Status
23	OCI image metadata labels (version, git commit, repository, build date)	Not implemented in this submission
24	Additional commit-SHA image tag (e.g. student-ml-api:92f4abc)	Not implemented in this submission
25	Docker layer-cache comparison (app.py-only vs requirements.txt change)	Not implemented in this submission

Recommendation: Part 23 (OCI labels) and Part 24 (commit-SHA tag) are low-effort, high-value additions to release.yml — both use information (git SHA, build date) already available to the workflow at build time — and would meaningfully strengthen the traceability story in Section 9 if time permits before final submission.

9. Traceability Matrix (Part 21)

The assignment requires the full chain Pull Request → Merge Commit → Git Tag → Docker Image → Digest to be documented per release. Both released versions are shown separately below for clarity (the single combined table in the original draft mixed the two releases together).

9.1 Release v1.0.0

Artifact Element	Value / Reference
Pull Request	PR #1 — "test: introduce deliberate failure" (feature/ci-failure-test)
Final commit on branch	276d3a5 — fix: correct health endpoint test
Git Release Tag	v1.0.0
Docker Image Tags	1.0.0, v1.0.0, latest (at time of v1.0.0 release)
Image Digest	sha256:5683bcbc5ec86d748f60f872436af1cc6c423beee7a8096edb89136000da6626
Registry Path	ghcr.io/umama123/student-ml-api

9.2 Release v1.1.0

Artifact Element	Value / Reference

Pull Request	PR #3 — "Create README.md for Student ML API"
Merge Commit SHA	d7c513b
Git Release Tag	v1.1.0
Docker Image Tags	1.1.0, latest (repointed from 1.0.0)
Image Digest	To be recorded via docker inspect --format='{{index .RepoDigests 0}}' ghcr.io/umama123/student-ml-api:1.1.0
Registry Path	ghcr.io/umama123/student-ml-api

This chain demonstrates that any running container can be traced backward to the exact reviewed Pull Request that produced it — satisfying the assignment's core traceability requirement.

10. Failure Analysis (Part 26)

The assignment requires deliberately reproducing and documenting at least two pipeline failures. One is fully evidenced below; a second should be added (see recommendation) before final submission to meet the "at least two" requirement in full.

10.1 Documented Failure — Failed Pytest Assertion

Symptom	GitHub Actions reported "All checks have failed" on Pull Request #1; both the pull_request and push triggers of the CI workflow show a red ✕
Root Cause	The /health test asserted data["status"] == "wrong" instead of the actual API response "healthy" — an intentional, incorrect assertion.
Evidence	Fig. 1 (Section 4.1) — commit 40dfe3b, "2 failing checks."
Correction	Commit 276d3a5, "fix: correct health endpoint test", restored the assertion to data["status"] == "healthy". Re-run of CI shows "All checks have passed" (Fig. 2).